Chapter 6
Boost C++ Libraries

Abstract In this chapter we discuss the Boost C++ API. Boost is a peer-reviewed C++ class library which implements many interesting and useful data structures and algorithms. In particular we discuss the use of Boost smart pointers, Boost asynchronous IO, and IO Streams. Boost also implements many data structures which are not present in the C++ standard library (e.g. bimap). Boost Graph Library (BGL) is presented with the help of real-life example. We compare Boost multi-threading and memory pool performance to APR. We discuss the integration of Python with C++ using Boost. We conclude the chapter with a discussion of Boost Generic Image Processing Library.

Contents

6.1 Boost smart pointer and memory pool **128**
6.2 Boost asio framework **131**
6.3 Boost data structures **134**
6.4 Boost Graph Library **135**
6.5 Boost Spirit Framework **138**
6.6 Boost multi-threading **140**
6.7 Boost Python integration **141**
6.8 Boost Generic Image Processing Library (GIL) **141**
6.9 Conclusion **143**

Boost is a collection of C++ libraries and header files which are peer-reviewed, portable, and work well with standard C++ libraries. Indeed, some of the Boost libraries have even become part of the new C++ standard. The main libraries present in Boost (as of version 1.4.2) are given below:

1. Template meta programming and C++ enhancements: these include (i) `any`, a generic container of single values of different types, (ii) `array`, STL compliant wrapper for fixed sized arrays, (iii) `bimap`, bidirectional map for C++, (iv) concept checking tools, (v) `foreach` in C++, (vi) `functional/hash` for TR1 C++,
2. Data structures: disjoint sets, date/time, dynamic bitset, property maps, unordered associative containers, universally unique identifier (UUID),
3. Asynchronous IO (asio): see Section 6.2 below,

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_6, © Springer Science+Business Media, LLC 2011

4. Memory Pool and Smart Pointer: flyweight pattern, memory pool, see Section 6.1 below, pointer container, serialization (see also XDR in Section 9.1.1), smart pointers,
5. Mathematics: linear algebra, quaternions, octonions, interval arithmetic, special functions (see also GNU scientific library, rational number class, uBLAS, in Section 16.5), generic image library,
6. Boost Graph Library (BGL): library for manipulating graphs (vertices and edges),
7. Lexical analysis and parsing: regular expression parsing, Spirit LL parser framework, tokenizer. An example of using Boost Spirit framework is given in Section 13.4.1.
8. Multi-threading: interprocess, memory mapped files, shared memory, system interface, threading library interface, timer functions, interface to MPI (see Section 12.3.1 for more details on Boost MPI),
9. CRC checksum: see Section 9.2 for a detailed example,
10. Python/C++ integration: see Section 9.7 for an example.

We discuss some of the important Boost C++ libraries below:

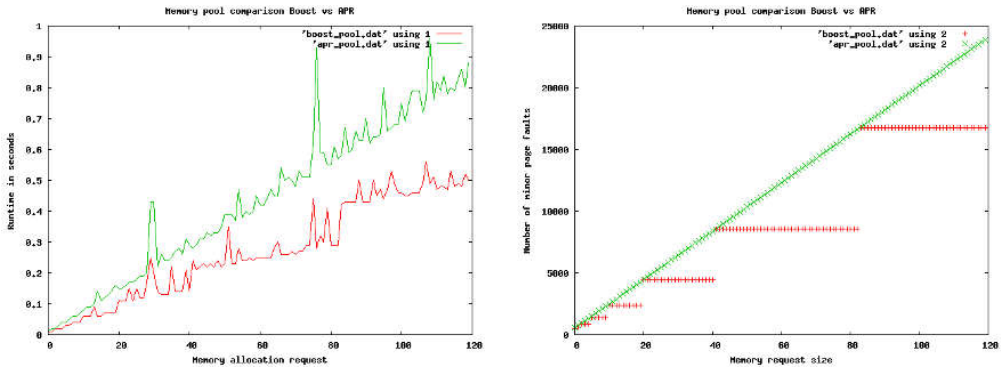
6.1 Boost smart pointer and memory pool

Memory pools are an important memory optimization technique. The `malloc` provided with the system library has been optimized for best average case performance; in many situations where the programmer is aware of the number, lifetime, usage patterns of objects, it is certainly possible to come up with allocation schemes which either reduce memory consumption, or optimize runtime (or both). We had previously seen the Apache Portable Runtime library (APR) which has a memory pool (see Section 5.4). It is instructive to compare performance of Boost memory pool with that of APR. See Figure 6.1(a) for a runtime comparison. Figure 6.1(b) denotes the number of minor page faults associated with the memory allocation. We will discuss performance optimization tools (perftools) in Section 7.1, and by linking against the provided `libtcmalloc` we reran the experiments. The comparison with and without `libtcmalloc` are shown in Figure 6.2(a), for Boost and Figure 6.2(b) for APR.

The internal memory allocation scheme for Boost and APR is readily apparent from the graphs.

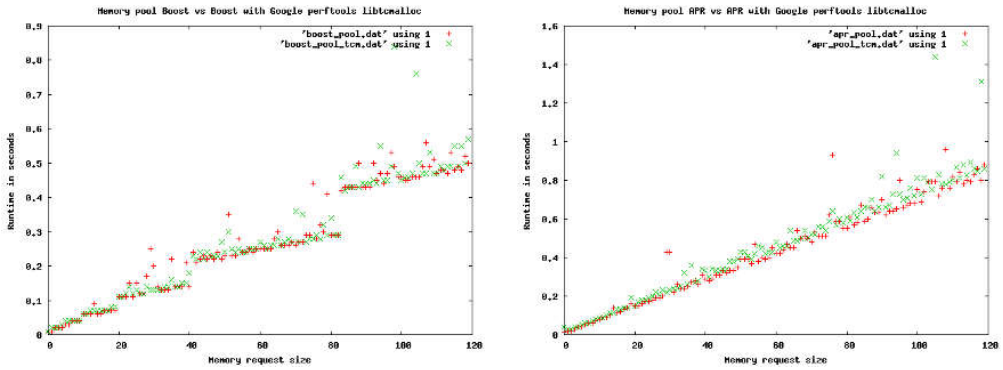
The source code for this example, which can also be used as a short model of using Boost pools and APR is given in Listing 6.1.

```
// \file compare_pool.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of Boost memory pool class
#include <fstream>
#include <boost/pool/pool.hpp>
#include <boost/pool/object_pool.hpp>
#include <apr_general.h>
#include <iostream>
#include <cassert>
```



(a) Runtime (b) No. minor faults

Fig. 6.1 Comparison of Boost vs APR memory pool



(a) Boost runtime (b) APR runtime

Fig. 6.2 Effects of linking with Google's perftools

```
10
   struct MyObject {
     int X;
     char *p;
     MyObject( ) { std::cout << __PRETTY_FUNCTION__ << "\n"; }
15   ~MyObject( ) { std::cout << __PRETTY_FUNCTION__ << "\n"; }
   };

   int normal_pool() {
     boost::pool<> pool( sizeof( double ) );
20
     double *allocated = (double*)pool.malloc();
   }

   int object_pool() {
25   boost::object_pool<MyObject> pool;

     MyObject *A = pool.malloc();
   }

30 unsigned long BoostPoolTest( unsigned long N ) {
     boost::pool<> pool( sizeof( int ) );
```

```
  unsigned long sum = 0;
  for( unsigned long i=0; i < N; ++i ) {
    int * const A = (int*) pool.malloc();
    *A = (int)A;
    sum += *A;
  }
  return sum;
}

unsigned long APRPoolTest( apr_pool_t *pool,
     unsigned long N ) {
  unsigned long sum = 0;
  for( unsigned long i=0; i < N; ++i ) {
    int * const A = (int*) apr_palloc( pool, sizeof( int ) );
    *A = (int)A;
    sum += *A;
  }
  return sum;
}
static size_t GetMemorySize(void)
{
   std::ifstream ifs;
   ifs.open("/proc/self/statm",std::ios::in);
   size_t memoryUsed = 0;
   ifs >> memoryUsed;
   return memoryUsed;
}

static void DisplayMemoryUsed(void)
{
   size_t memUsed = GetMemorySize();
   std::cout << std::endl << "Program consumed "<< memUsed << " kb.\n";
}

int main(int argc, char *argv[] ) {
  static const unsigned long COUNT = atol( argv[1] );
  int use_boost = atoi( argv[2] );
  apr_status_t retval;
  apr_pool_t *apr_pool;
  normal_pool();
  object_pool();
  return 0;
  if( use_boost ) {
    BoostPoolTest( COUNT );
  } else {
    apr_initialize();
    apr_pool_create( &apr_pool, NULL );
    APRPoolTest( apr_pool, COUNT );
  }
  //DisplayMemoryUsed();
  return 0;
}
```

Listing 6.1 Example of Boost memory pool

One important thing to keep in mind when using Boost `object_pool` is that the *destructor* of the object in the pool is called automatically when the pool is destroyed. In the above listing we see:

`MyObject::~MyObject()`

when the pool goes out of scope.

6.2 Boost asio framework

A fundamental component of the Boost `asio` framework for communication is the *socket* which provides an abstraction for the usual networking socket available in POSIX. The socket as defined in `basic_datagram_socket` is a template type which can be instantiated with different protocols. The socket type can be configured to specify the endpoint-type and other parameters. It supports functions such as `bind`, `connect`, `close`, `receive`, `receive_from` and `shutdown`.

Consider a UDP server returning the server's date and time on a specified port. The code using Boost asio is shown in Listing 6.2. The corresponding client code is shown in Listing 6.3. We can trivially change the server code to instead return the load average as shown.

```
// \file udp_server.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of Boost ASIO UDP connection
#include <ctime>              // for the date/time function
#include <iostream>           // program IO
#include <fstream>            // for ifstream
#include <string>             // string to store date
#include <boost/array.hpp>    //
#include <boost/asio.hpp>

static const int PORT_NUMBER = 8954; // port of server
using boost::asio::ip::udp;

#ifdef PRINT_TIME
std::string make_info_string() {
  using namespace std; // For time_t, time and ctime;
  time_t now = time(0);
  return ctime(&now);
}
#endif
std::string make_info_string() {
  char temp[1024];
  std::ifstream ifs("/proc/loadavg");
  ifs.getline( temp, 1024 );
  return std::string( temp );
}

int main() {
  boost::asio::io_service io_service;

  udp::socket socket(io_service, udp::endpoint(udp::v4(), PORT_NUMBER));
  while( true ) {
    boost::array<char, 1> recv_buf; // this is just the trigger
    udp::endpoint remote_endpoint;
    boost::system::error_code error;
    socket.receive_from(boost::asio::buffer(recv_buf),
                        remote_endpoint, 0, error);
    std::string message = make_info_string();
    boost::system::error_code ignored_error;
    socket.send_to(boost::asio::buffer(message),
                   remote_endpoint, 0, ignored_error);
  }
  return 0;
}
```

Listing 6.2 Example of Boost asio UDP server

```
// \file udp_client.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of Boost ASIO UDP connection
#include <iostream>           // for program IO
#include <stdlib.h>           // exit
#include <string>             // hostname
#include <boost/array.hpp>    // boost::array
#include <boost/asio.hpp>     // Boost ASIO library

using boost::asio::ip::udp;

int main(int argc, char* argv[]) {
  std::string hostName;
  if (argc != 2) {
    std::cerr << "Usage: udp_client <host>" << std::endl;
    exit( 1 );
  } else {
    hostName = argv[1];
  }
  boost::asio::io_service io_service;
  udp::resolver resolver(io_service);
  // the port number is specified as a string argument
  udp::resolver::query query(udp::v4(), hostName, "8954");
  udp::endpoint receiver_endpoint = *resolver.resolve(query);

  udp::socket socket(io_service);
  socket.open(udp::v4());           // Use UDP socket

  boost::array<char, 1> send_buf  = { 0 };
  socket.send_to(boost::asio::buffer(send_buf), receiver_endpoint);

  boost::array<char, 128> recv_buf;
  udp::endpoint sender_endpoint;
  size_t len = socket.receive_from(
           boost::asio::buffer(recv_buf),
           sender_endpoint);
  if( len > 128 ) len=128;
  std::cout.write(recv_buf.data(), len);
  return 0;
}
```

Listing 6.3 Example of Boost asio UDP client

Compiling this requires the `boost_system` and `pthread` library to be linked:

```
$g++ -o udp_server udp_server.cpp  -lboost_system -lpthread
```

6.2.1 Boost IOStreams framework

The architecture of Boost IOStreams is based on a pipeline of components in which data is passed from *source* to *sink* and undergoes filtering and conversion at each stage of the pipeline. The source for the stream can be a disk file, memory mapped file, or array of bytes. The sink can be a disk file, memory-mapped file, or array of bytes as well. The filters included in the IOStreams library include compression filters based on `zlib`. We present an example of a writer and reader based on Boost IOStreams which store the data on disk in a compressed manner.

The writer is shown in Listing 6.4, while the reader is shown in Listing 6.5.

```
// \file file_io_write.cpp
// \author Sandeep Koranne, (C) 2010
// \description Using Boost IO Streams
#include <boost/iostreams/device/file.hpp>
#include <boost/iostreams/filtering_stream.hpp>
#include <boost/iostreams/filter/gzip.hpp>
using namespace boost::iostreams;
int main() {
  filtering_ostream out; // STL compliant io stream
  out.push(gzip_compressor()); // step 1 : compress
  out.push(file_sink("am.txt"));// step 2: write to disk
  out << 1 << " " << 2 << " " << 3 << " " << "Johnson";
  return 0;
}
```

Listing 6.4 Example of Boost IOStreams writer

```
// \file file_io_read.cpp
// \author Sandeep Koranne, (C) 2010
// \description Using Boost IO Streams
#include <string>
#include <iostream>
#include <boost/iostreams/device/file.hpp>
#include <boost/iostreams/filtering_stream.hpp>
#include <boost/iostreams/filter/gzip.hpp>
using namespace boost::iostreams;
int main() {
  filtering_istream data_in; // STL iostream compliant
  data_in.push(gzip_decompressor()); // step 2 : decompress
  data_in.push(file_source("am.txt")); // step 1: read from source
  int x,y,z;  data_in >> x >> y >> z;
  std::string name;  data_in >> name;
  std::cout << "Read " << x << " " << y << " " << z
            << " Name = " << name << std::endl;
  return 0;
}
```

Listing 6.5 Example of Boost IOStreams reader

Compiling and running these programs gives:

```
$g++ file_io_write.cpp -o file_io_write -lboost_iostreams -lz
./file_io_write
$file am.txt
am.txt: gzip compressed data
$g++ file_io_read.cpp -o file_io_read -lboost_iostreams -lz
$./file_io_read
Read 1 2 3 Name = Johnson
```

Later in Section 8.1 we shall see more examples of using `zlib` for compression and decompression.

6.3 Boost data structures

6.3.0.1 Dynamic bitset

Boost dynamic bitset provides a class to represent a set of bits which can be accessed using the `[]` operator. It is identical to the `std:bitset`, except that the size of underlying container is not fixed at compile time (for `std::bitset` it has to set at compile time).

6.3.0.2 Bimap: bidirectional map

Boost Bimap is a map which represents bidirectional relations between elements. This container is designed to work as two independent maps, from X to Y, and Y to X can be written as:

```
typedef boost::bimap<X,Y> XY_BM;
```

For a given bimap, the left map view represents the X to Y map, while the right map view represents the Y to X map. For example a representation for a dictionary can be:

```
typedef boost::bimap< std::string, int > DICT;
DICT d;
d.insert( DICT::value_type("Jack", 1 ) );
```

A full example is given in Listing 6.6.

```
// \file bimap_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of Boost bimap
#include <boost/bimap.hpp>
#include <string>
#include <iostream>

typedef boost::bimap< std::string, int > DICT;

int main( int argc, char * argv[] ) {
  DICT d;
  d.insert( DICT::value_type("Jack", 1 ) );
  d.insert( DICT::value_type("Jane", 2 ) );
  d.insert( DICT::value_type("Jill", 3 ) );
  for( DICT::const_iterator it = d.begin();
       it != d.end(); ++it ) {
    std::cout << it->left << "\t" << it->right << std::endl;
  }
  std::cout << "-------\n";
  for( DICT::left_map::const_iterator it=d.left.begin();
       it != d.left.end(); ++it ) {
    std::cout << it->first << "\t" << it->second << std::endl;
  }
  std::cout << "-------\n";
  for( DICT::right_map::const_iterator it=d.right.begin();
       it != d.right.end(); ++it ) {
    std::cout << it->first << "\t" << it->second << std::endl;
  }
  return 0;
```

```
30 }
```

Listing 6.6 Example of Boost bimap container

Compiling and running this program gives us:

```
Jack 1
Jane 2
Jill 3
-------
Jack 1
Jane 2
Jill 3
-------
1 Jack
2 Jane
3 Jill
```

6.3.0.3 Array: STL compliant container for fixed size array

```
boost::array< int, 1024 > integer_buffer;
```

By providing the type and the length of the array as the template parameter Boost array provides an abstraction which is STL compliant.

6.4 Boost Graph Library

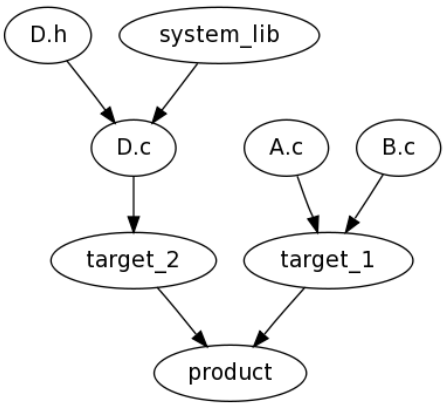


Fig. 6.3 Graph of dependencies using BGL

We present an example of using Boost Graph Library to solve a real world problem. Consider the problem of dependency analysis of programs. Given the input *make* type file:

```
begin
D.c      : D.h system_lib ;
target_1 : A.c B.c ;
product  : target_1 target_2 ;
target_2 : D.c ;
end
```

the system has to compute the correct order in which to process the files. The graph of dependencies is shown in Figure 6.3. The C++ program using BGL is shown in Listing 6.7.

```
// \file bgl_make.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of BGL for dependency analysis

#include <iostream>           // messaging
#include <fstream>            // reading Make file
#include <cassert>            // assertions
#include <vector>             // vertices
#include <list>               // edges
#include <deque>              // topological sort
#include <string>             // string
#include <sstream>            // string parsing
#include <map>                // string->id
#include <boost/graph/vector_as_graph.hpp> // BGL
#include <boost/graph/topological_sort.hpp>// topological_sort

typedef std::map< std::string, int > FileTable;
FileTable gFileTable;

std::vector< std::string > gFileNames;

static int LookupOrAdd( const std::string& Name ) {
  int retval = gFileTable.size();
  if( gFileTable.find( Name ) == gFileTable.end() ) {
      gFileTable[ Name ] = retval;
      gFileNames.push_back( Name );
      return retval++;
  } else {
    return gFileTable[ Name ];
  }
}

static const size_t MAX_N = 100;
typedef std::vector< std::list< int > > Graph;
Graph gDependency( MAX_N );
// the input format is simple
// target : <dep1> <dep2> <dep3>

static void ParseFileAndCreateGraph( const std::string& fileName ) {
  std::ifstream ifs( fileName.c_str() );
  if( !ifs ) { std::cerr << "Unable to open file: " << fileName << std::endl;
    exit( 1 );
  }
  char a_line[ 1024 ];
  ifs.getline( a_line, 1024 );
  std::string line( a_line );
  if( line != "begin" ) {
    std::cerr << "File: " << fileName << " is not a .mak file\n";
    exit( 1 );
  }
  while( true ) { // parse the remaining file
    ifs.getline( a_line, 1024 );
    std::string line( a_line );
    if( line == "end" ) break;
```

```
    // actual parsing
    std::istringstream sis;
    sis.str( line );
    std::string target; sis >> target;
    std::string token; sis >> token;
    if( token != ":" ) {
      std::cerr << "Parsing error: " << token << " expecting : " << std::endl;
      exit( 1 );
    }
    int target_id = LookupOrAdd( target );

    while( sis ) {
      sis >> token;
      if( token == ";") break;
      int dep_id = LookupOrAdd( token );
      gDependency[ target_id ].push_back( dep_id );
    }
  }
}

std::deque<int> t_order;
static void ComputeDependency(void) {
  boost::topological_sort(
    gDependency, std::front_inserter( t_order ),
    boost::vertex_index_map( boost::identity_property_map() ) );
}

static void PrintDependency(void) {
  for( std::deque<int>::reverse_iterator it=t_order.rbegin();
       it != t_order.rend(); ++it ) {
    if( *it >= gFileNames.size() ) continue;
    std::cout << "Action " << gFileNames[*it] << std::endl;
  }
  std::cout << std::endl;
}

int main( int argc, char * argv[] ) {
  std::string MakeFileName( argv[1] );
  ParseFileAndCreateGraph( MakeFileName );
  ComputeDependency();
  PrintDependency();
  return 0;
}
```

Listing 6.7 Example of BGL

The main features of Boost Graph Library are:

- Graph construction: in BGL graphs can be *directed* or *undirected*. Once a graph has been constructed, vertices and edges can be added into it. The underlying data-structure for the graph depends on the application, and determines the run-time complexity of the algorithms using the graph. Graphs can be dense (with a high ratio of edges (M to N^2), where M denotes the number of edges, and N denotes the number of vertices. For sparse graphs (such as those found in large networks), a list of nodes per vertex (called an adjacency list) implementation is efficient. The functions `add_vertex` adds and returns an opaque handle to the newly added vertex. Similarly, the `add_edge` function returns a 2-tuple, a handle to the edge, and a Boolean to specify whether the edge was actually inserted or was already present.
- Vertex and Edge descriptors: access to the graph's nodes and edges is made through opaque handles called *descriptors*. Graphs may use arbitrary types for

handles, although integers and pointers are common. The descriptor of a graph are accessible using the `graph_traits` class. Vertex descriptors can only be constructed, copied and compared for equality. Edge descriptors can return the constituent vertex descriptors.

- Property maps: property maps are a generic method to attach auxiliary information to vertices and edges. In most examples the vertices represent and model some parameter in the system. Without property maps, the programmer must ensure that the graph, and the model remain in sync, but by using property maps (which can be as simple as a 1-1 index into the model), this problem is removed. The 1-1 map is called the *identity map*, and is a common property map for integer typed vertices. BGL has a number of predefined property types, such as `vertex_name_t`, to install an integer as name for each vertex, `property< vertex_name_t, int>` Multiple property types can be added to vertices and edges using the third parameter of `property` which allows for chaining.
- Algorithms on graphs: Boost graph library has implemented a number of useful graph algorithms which are immediately available to all users of BGL. The algorithms are based on (i) graph traversal, (ii) graph search, and (iii) graph manipulation. Graph traversal is performed using *iterators*, BGL has iterators for vertices and edges. An example of using graph iterators:

  ```
  typedef typename graph_traits<G>::vertex_iterator VIT;
  std::pair< VIT, VIT > v_of_g = vertices(g);
  for( ; v_of_g.first != v_of_g.second; ++v_of_g.first )
  ```

  The `vertices(Graph)` function returns a 2-tuple (called a *tie*) as a pair of vertex iterators. The first points to a valid vertex, while the second points to the *end* of the vertex iterator sequence. The algorithms implemented in BGL include:

  1. Breadth first search (BFS) and Depth first search (DFS),
  2. Shortest path,
  3. Minimum spanning tree (MST),
  4. Connected components,
  5. Maximum flow,
  6. Graph search,

- Adapters: graph adapters in BGL are filters on the input graph which produce another BGL graph (through a transformation), or produce a graph in an external format such as DOT (using Graphviz .dot, see Section 19.4), or LEDA graphs, or Stanford GraphBase format.

6.5 Boost Spirit Framework

The Spirit Parser Framework is an object oriented recursive descent parser generator framework implemented using template meta-programming techniques. Expression templates allow users to approximate the syntax of Extended Backus Naur Form

(EBNF) completely in C++. Parser objects are composed through operator overloading and the result is a backtracking LL(∞) parser that is capable of parsing rather ambiguous grammars. Spirit can be used for both lexing and parsing, together or separately.

Consider a parser modeled after the classic recursive descent method for parsing mathematical expressions of the form:

```
start <- assignment;
assignment <- lhs '=' rhs;
lhs <- literal;
rhs <- expression;
expression <- '(' expression ')' | term op expression;
op <- XOR | NEG;
term <- term AND factor;
factor <- literal | literal OR factor;
```

The SPIRIT implementation of this parser is shown below:

```
struct EqnParser : public grammar<EqnParser> {
  template <typename ScannerT>
  struct definition  {
    definition(EqnParser const& /*self*/)  {
      expression
        = str_p("INPUT()") [&do_input]
        | str_p("OUTPUT()")[&do_output]
        | term
        >> *(   ('#' >> term)    [&do_or]
                |   ('$' >> term)[&do_xor]
                |   ('-' >> term)[&do_subt]
                )
        ;

      term
        =   factor
        >> *(   ('&' >> factor)[&do_and]
                |   ('/' >> factor)[&do_div]
                )
        ;

      factor
        =   lexeme_d[(+digit_p)[&do_int]]
        |   lexeme_d[(+(alnum_p|'_'|'['|']'))[&do_literal]]
        |   '(' >> expression[&do_expr] >> ')'
        |   ('!' >> factor)[&do_neg]
        |   ('#' >> factor)
        ;

      assign
        =   lexeme_d[(+(alnum_p|'_'|'['|']'))[&do_lhs]]
        >> '=' >> expression[&do_final] >> ';'
        ;
    }
```

```
    rule<ScannerT> expression, term, factor, assign;
    rule<ScannerT> const&
    start() const { return assign; }
  };
};
```

Using Boost SPIRIT framework, the formal grammar is translated to C++ code using the following conventions: (i) production choices are delineated using the `|` operator, (ii) Kleene star closures are defined using the `*` operator, (iii) the stream redirection operator `>>` is overloaded to define production rules, and (iv) the array index operator `[]` is overloaded to define the function to be called when the production is matched. In the above example, we see that when the operator `#` is matched for example, the `do_or` function is called.

The function to be called is defined to accept the *left* and *right* operand, e.g.:

```
void do_xor(char const* l, char const* r) {
  // take the 2 top elements from the stack
  // and make their tree and add to sub tree
  Tree* pT = new Tree( XOR, GetName() );
  pT->left = treeStack.top();   treeStack.pop();
  pT->right = treeStack.top();  treeStack.pop();
  treeStack.push(pT);
}
```

For more details on parsing see Section 13.4.1.

6.6 Boost multi-threading

The `boost::thread` class is responsible for multi-threading in Boost C++ programs. The Boost threading interface provides an operating system independent threading API which is compatible with C++ programming idioms. Threads can be created with user defined functions and class member functions. The usual POSIX threading capabilities of critical section protection, using *mutex*, thread local variables, and *pthread once* functions are also supported. An example of using Boost threads with member functions representing the work function is shown in Listing 6.8.

```
// \file boost_thread_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of using Boost threading API
#include <iostream>           // for Program IO
#include <cassert>            // assertion checking
#include <boost/thread/thread.hpp> // Boost threads

struct MyComputation {
  int how_many;
  void operator()(void) {
    std::cout << "Thread function called : " << how_many << "\n";
  }
  MyComputation( int x ): how_many( x ) {}
};

int main( int argc, char* argv[] ) {
  MyComputation C1(10), C2(20);
```

```
  boost::thread A( C1 ), B( C2 );

  A.join();
  B.join();

  std::cout << std::endl;
  return (0);
}
```

Listing 6.8 Example of using Boost threads

6.7 Boost Python integration

Boost Python library is a C++ framework for interfacing C++ functions and objects with the Python language. Using the Boost Python module arbitrary C++ functions and objects can be interfaced with Python. Consider the following example:

```
// C++ code
char* PrintHello(void) {
  return ``Hello, World!'';
}
#include <boost/python.hpp>
BOOST_PYTHON_MODULE( hello_ext )
{
  using namespace boost::python;
  def(``hello_world'', PrintHello);
}
```

Now, we can import this module in Python and issue this function call from Python.

6.8 Boost Generic Image Processing Library (GIL)

Boost also has a generic image processing library (GIL) which implements image processing functions such as gradient calculation, color space conversion, image transforms, as well as reading and writing common image file formats such as JPEG. An example of using Boost GIL to compute the histogram of a given JPEG file is shown in Listing 6.9. The output histogram is shown in Figure 6.4.

```
// \file gil_hist_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of using Boost GIL (generic image library)
#include <iostream>           // for program IO
#include <iterator>           // ostream iterator
#include <fstream>            // reading image files.
#include <cstdlib>            // for exit
#include <boost/gil/image.hpp> // GIL image
#include <boost/gil/typedefs.hpp> // GIL typedefs
#include <boost/gil/extension/io/jpeg_io.hpp> // reading JPEG files.

using namespace boost::gil;

template <typename GrayView, typename R>
void gray_image_hist(const GrayView& img_view, R& hist) {
```

```
  for (typename GrayView::iterator it=img_view.begin(),
   en=img_view.end(); it != en; ++it)
    ++hist[ *it ];
}

template <typename V, typename R>
void get_hist(const V& img_view, R& hist) {
    gray_image_hist(color_converted_view<gray8_pixel_t>(img_view), hist);
}

int main( int argc, char *argv[] ) {
  if( argc != 2 ) {
    std::cerr << "Usage: gil_hist_example <file.jpg>...\n";
    exit(1);
  }
  rgb8_image_t img;
  jpeg_read_image( argv[1], img);
  std::vector<int> histogram(256, 0);

  get_hist( const_view(img),histogram );
  std::fstream output_file( "output.txt", std::ios::out );
  std::copy( histogram.begin(), histogram.end(),
       std::ostream_iterator<int>( output_file, "\n" ));
  output_file.close();
  std::cout << std::endl;
  return (0);
}
```

Listing 6.9 Example of using Boost GIL

In addition to reading JPEG files, GIL also supports PNG (portable network graphics) and TIFF input and output. Using these IO functions pixel data from JPEG, PNG files can be read into an `Image` data-structure in memory. Thereafter, calculations and transforms can be applied to the in-memory representation of the image. The transformed image can be written back to disk in JPEG, TIFF or PNG format.

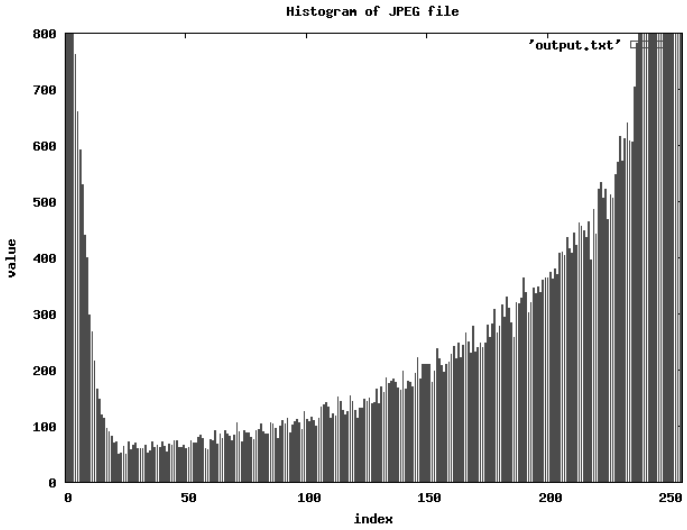


Fig. 6.4 Boost Generic Image Library (GIL) computing histograms of JPEG files

Boost GIL supports many different types of image representations, including: (i) planar and interleaved, (ii) color space and alpha channel, (iii) multi-depth channels, of 8-bit, 16-bit and 24-bit colors, (iv) multi-ordered channels, RGB vs BGR, and (v) row alignment policy.

6.9 Conclusion

Boost C++ libraries provide many well designed data structures and application framework tools, which are useful in a broad range of applications. Using Boost libraries, not only can the developer reuse high-quality code, but also adopt an operating system independent approach. Boost APIs have been effectively used in many scientific and engineering applications, and with the continued development of Boost C++ library, this trend is only expected to increase.